PROJECT INSTRUCTION GUIDE

Renewable Energy
Output Prediction
Using Artificial Neural Networks

Reference: Negnevitsky, M. (2005). Artificial Intelligence: A Guide to Intelligent Systems, 2nd Ed.
Chapter 6: Artificial Neural Networks

Intelligent Systems Course • Instructor Resource • A.Y. 2025

1. Project Overview

This project guides students to apply Artificial Neural Networks (ANN) — as covered in Chapter 6 of Negnevitsky's Artificial Intelligence textbook — to a real-world energy problem: predicting the output of solar and wind energy systems from weather data. Students will design, train, and evaluate a multilayer neural network using backpropagation learning.

Project at a Glance

- Topic: Renewable Energy — Solar & Wind Output Prediction
- AI Technique: Multilayer Artificial Neural Networks (Backpropagation)
- Goal: Predict energy output (kWh) from weather input features
- Datasets: PVGIS, NREL NSRDB, Global Wind Atlas, UCI Energy Efficiency
- Duration: 2 weeks | Output: Python Code + Report + Presentation Slides

1.1 Background & Motivation

Solar and wind energy output is highly variable, it depends on cloud cover, temperature, wind speed, humidity, and time of day. Traditional statistical models (like linear regression) cannot capture these complex non-linear relationships. Artificial Neural Networks, which mimic the learning process of biological brains, are ideal for this task: they can learn patterns from historical data and generalize to make accurate predictions on new data.

1.2 Learning Objectives

- **Understand** the structure and function of biological and artificial neurons (Negnevitsky §6.1–6.2)
- **Implement** a multilayer perceptron (MLP) with input, hidden, and output layers (§6.4)
- **Apply** the backpropagation learning algorithm to train the network on real energy data (§6.4–6.5)
- **Evaluate** model performance using RMSE, MAE, and R^2 metrics
- **Compare** ANN prediction accuracy against a traditional statistical baseline

2. Tools & Technologies Required

2.1 Programming Language

Python 3.9 or higher is required. Students may use Google Colab (free, no installation needed) or Jupyter Notebook locally.

2.2 Required Python Libraries

Install all libraries before starting the project

- numpy — numerical computations and array operations
- pandas — data loading, cleaning, and manipulation
- matplotlib / seaborn — data visualization and result plots
- scikit-learn — data preprocessing (scaling, splitting) and evaluation metrics
- tensorflow or pytorch — for building and training the neural network
- requests — for fetching data from PVGIS / NASA POWER APIs

Note: *Google Colab already has TensorFlow and most libraries pre-installed. Recommended for beginners: use Colab + Keras (TensorFlow's high-level API) for building the ANN.*

2.3 Development Environment

- **Google Colab** — free cloud Jupyter environment with GPU support (recommended)
- **Jupyter Notebook** — local environment for offline development
- **VS Code + Python extension** — for writing modular Python scripts

3. Datasets to Use

Students must use at least two datasets. Below are all recommended datasets with download instructions and their specific role in the ANN project:

Dataset	Source	Key Variables	Role in Project
PVGIS Solar Data	**European Commission (free)**	Solar irradiance, PV output, temp, tilt, location	Primary target variable — solar energy output (kWh)
NREL NSRDB	**US Nat'l Renewable Energy Lab**	GHI, DNI, DHI, wind speed, humidity, cloud cover (hourly)	ANN input features for solar and weather patterns
Global Wind Atlas	**DTU / World Bank (free)**	Wind speed at hub height, direction, capacity factor	Wind energy output target for ANN wind model
UCI Energy Efficiency	**Kaggle / UCI ML Repo**	Building features, heating/cooling load, energy use	Supplementary — energy demand context

Note: *PVGIS API: Access at https://re.jrc.ec.europa.eu/pvg_tools/en/ — enter your city coordinates, select hourly data, download CSV. NREL NSRDB requires a free API key at https://developer.nrel.gov*

3.1 Data Preprocessing Steps

1. Download hourly solar/wind data for at least 1 full year for your chosen location
2. Load with pandas: df = pd.read_csv('solar_data.csv') — check shape and column names
3. Handle missing values: df.dropna() or df.fillna(df.mean()) for gaps in sensor data
4. Create time features: extract hour, day_of_week, month, season from the datetime column
5. Normalize inputs using MinMaxScaler: scale all features to range [0, 1]
6. Split data: 70% training, 15% validation, 15% testing (use chronological split, not random)
7. Define input features (X) and target variable (y = energy output in kWh)

Note: *IMPORTANT: Use a time-based split (not random) to avoid data leakage — future data must not appear in the training set.*

4. Neural Network Design

This section explains how to design your ANN model following the concepts in Negnevitsky Chapter 6. Students should understand each component before coding.

4.1 Network Architecture

Recommended ANN Architecture for Energy Prediction

- Input Layer: 5 neurons — one per feature (irradiance, temp, wind speed, humidity, hour)
- Hidden Layer 1: 10 neurons — ReLU activation function
- Hidden Layer 2: 8 neurons — ReLU activation function
- Output Layer: 1 neuron — Linear activation (regression output = kWh value)
- Loss Function: Mean Squared Error (MSE)
- Optimizer: Adam (adaptive learning rate — improves on basic gradient descent)

4.2 The Neuron — From Negnevitsky §6.2

Each artificial neuron receives weighted inputs, sums them, adds a bias, and passes the result through an activation function. The network learns by adjusting these weights during training. Refer to Figure 6.3 in Negnevitsky for a diagram of the neuron model.

4.3 Backpropagation Learning — From Negnevitsky §6.4

Backpropagation is the core training algorithm for multilayer networks. It works in two passes:

8. Forward pass: Input data flows through the network, producing a predicted output
9. Error calculation: Compare prediction to actual value using loss function (MSE)
10. Backward pass: Error is propagated back, computing gradient for each weight
11. Weight update: Weights adjusted in the direction that reduces error (gradient descent)
12. Repeat for all training samples across multiple epochs until error converges

4.4 Recommended Training Parameters

- **Epochs:** 100–200 (monitor validation loss to detect overfitting)
- **Batch size:** 32 samples per gradient update
- **Learning rate:** 0.001 (Adam optimizer default — good starting point)
- **Early stopping:** Stop training if validation loss does not improve for 10 epochs
- **Dropout:** Optional — 20% dropout rate between layers to prevent overfitting

Note: *Plot training vs. validation loss curves per epoch to visually detect overfitting. If validation loss rises while training loss falls, reduce model complexity or add dropout.*

5. Step-by-Step Project Instructions

Students must follow these 8 steps in sequence over the 8-week project period:

Step	Task	Description	Output
1	**Problem Definition**	Write a 1-page proposal: define the prediction target (solar vs wind), location/country, chosen dataset, and expected ANN architecture.	*Project Proposal*
2	**Environment Setup**	Set up Python environment (Colab or local). Install all libraries. Create a GitHub repository with a README file.	*Working environment + GitHub repo*
3	**Data Collection**	Download at least 1 year of hourly solar/wind data. Fetch weather data from PVGIS or NREL API for your chosen region.	*Raw dataset CSV files*
4	**EDA & Preprocessing**	Explore data: plot distributions, correlations, seasonal trends. Clean, normalize, and split into train/validation/test sets.	*Cleaned dataset + EDA report with plots*
5	**ANN Implementation**	Build the multilayer ANN in Python (Keras/TensorFlow). Define architecture, compile with MSE loss and Adam optimizer, train model.	*ann_model.py + training notebook*
6	**Training & Tuning**	Train for 100–200 epochs. Plot loss curves. Tune hidden layers, neurons, or dropout if overfitting. Save the best model.	*Trained model file (.h5) + loss plots*
7	**Evaluation & Comparison**	Evaluate on test set: compute RMSE, MAE, R^2. Plot predicted vs actual. Compare ANN to a linear regression baseline.	*Results tables + comparison charts*
8	**Report & Presentation**	Write final report (see outline below). Prepare 7–10 slides for presentation. Be ready to demo model predictions live.	*Final Report + Slides*

6. Final Report Outline

The final report should be 10–15 pages (excluding appendices). Follow this structure:

Section 1 — Introduction (1–2 pages)

- **Why renewable energy prediction matters for grid stability and sustainability**
- **Research question: Can ANN predict solar/wind output accurately from weather data?**
- **Overview of the chosen dataset and geographic location**
- **Scope: which energy type (solar, wind, or both) is being predicted**

Section 2 — Literature Review (1–2 pages)

- **Summary of Negnevitsky Ch. 6: neurons, perceptrons, multilayer networks, backpropagation**
- **Related studies: ANN applied to energy forecasting (cite 2–3 recent papers)**
- **Comparison: why ANN outperforms linear regression for this problem**

Section 3 — Methodology (3–4 pages)

- **Dataset description: source, time period, location, variables, resolution**
- **Preprocessing steps: missing values, feature engineering, normalization, split**
- **Network architecture diagram (draw or screenshot from Keras model summary)**
- **Training setup: loss function, optimizer, hyperparameters, epochs**
- **Evaluation metrics used: RMSE, MAE, R^2 — with formulas**

Section 4 — Results (2–3 pages)

- **Training and validation loss curves (plot per epoch)**
- **Predicted vs. actual output plot for the test period**
- **Performance metrics table: RMSE, MAE, R^2 for ANN and baseline**
- **Error analysis: when does the model perform poorly? (cloud cover, night hours?)**

Section 5 — Discussion (1–2 pages)

- **Did the ANN achieve good prediction accuracy? What RMSE is acceptable?**
- **Which input features were most important for prediction?**
- **Limitations: data quality, generalizability to other locations, model simplicity**
- **Recommendations: what would improve the model? (LSTM, more data, more features)**

Section 6 — Conclusion & References

- **2–3 paragraph summary of findings and contribution**
- **APA/IEEE references — must include Negnevitsky (2005) and dataset sources**

7. Deliverables & Timeline

The following must be submitted by the end of the 8-week period:

#	Deliverable	Timeline	Description
1	**Project Proposal**	Week 1–2	*1–2 pages: problem statement, ANN method choice, datasets, expected results*
2	**Dataset & EDA Report**	Week 3	*Cleaned datasets with exploratory analysis, correlation plots, seasonal patterns*
3	**ANN Implementation Code**	Week 4–6	*Python code: data pipeline, network architecture, training loop, validation*
4	**Prediction Results**	Week 7	*Plots of ANN predictions vs actual output; RMSE, MAE, R^2 scores*
5	**Final Project Report**	Week 8	*Full written report: intro, methodology, results, discussion, conclusion*
6	**Presentation Slides**	Week 8	*7–10 slides summarizing the project (template provided by instructor)*

7.1 Grading Criteria

Suggested grading breakdown for instructors

- Dataset & EDA (15%) — quality of data collection, cleaning, and visualizations
- ANN Implementation (35%) — correct architecture, proper training setup, clean code
- Results & Evaluation (25%) — accuracy metrics, plots, comparison vs baseline
- Final Report (15%) — clarity, structure, proper textbook citation
- Presentation & Defense (10%) — clarity, ability to explain ANN to classmates

8. Starter Code Template

Students should use this template as the skeleton for their ann_model.py file. Replace the TODO sections with their own implementation:

```
# ── Renewable Energy ANN — Starter Code ──────────────────────
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, r2_score
from tensorflow import keras

# ── Step 1: Load & Preprocess Data ──────────────────────────
df = pd.read_csv('solar_data.csv', parse_dates=['datetime'])
df['hour'] = df['datetime'].dt.hour
df['month'] = df['datetime'].dt.month
features = ['irradiance','temperature','wind_speed','humidity','hour']
X = df[features].values
y = df['energy_kwh'].values.reshape(-1, 1)

scaler_X = MinMaxScaler(); scaler_y = MinMaxScaler()
X_scaled = scaler_X.fit_transform(X)
y_scaled = scaler_y.fit_transform(y)

# ── Step 2: Build ANN Model ─────────────────────────────────
model = keras.Sequential([
    keras.layers.Dense(10, activation='relu', input_shape=(5,)),
    keras.layers.Dense(8,  activation='relu'),
    keras.layers.Dense(1,  activation='linear')  # regression output
])
model.compile(optimizer='adam', loss='mse', metrics=['mae'])

# ── Step 3: Train ───────────────────────────────────────────
history = model.fit(X_train, y_train, epochs=150,
    batch_size=32, validation_data=(X_val, y_val), verbose=1)

# ── Step 4: Evaluate ────────────────────────────────────────
y_pred = scaler_y.inverse_transform(model.predict(X_test))
y_true = scaler_y.inverse_transform(y_test)
rmse = np.sqrt(mean_squared_error(y_true, y_pred))
print(f'RMSE: {rmse:.3f} | R²: {r2_score(y_true,y_pred):.3f}')
```

> **Note:** *Students should experiment with different hidden layer sizes and activation functions. Challenge: try adding a third hidden layer or using LSTM for time-series prediction.*

9. References

1. Negnevitsky, M. (2005). Artificial Intelligence: A Guide to Intelligent Systems (2nd ed.). Pearson/Addison-Wesley. [Chapter 6: Artificial Neural Networks]

2. PVGIS Photovoltaic Geographical Information System. European Commission. Retrieved from https://re.jrc.ec.europa.eu

3. NREL National Solar Radiation Database (NSRDB). National Renewable Energy Laboratory. Retrieved from https://nsrdb.nrel.gov

4. Global Wind Atlas. Technical University of Denmark / World Bank Group. Retrieved from https://globalwindatlas.info

5. Dua, D. & Graff, C. (2019). UCI Machine Learning Repository. University of California, Irvine. Retrieved from https://archive.ics.uci.edu/ml

6. Chollet, F. et al. (2015). Keras. TensorFlow. Retrieved from https://keras.io

Good luck to your students!

The best way to learn AI is to apply it to problems that matter.